RAG-Enabled Multi-Agent Digital Twin for O-RAN: Autonomous Fidelity-Driven Self-Adaptation

Varun A L, Harieharan R, Tarakesh M, Jai Adithiya R, and Mithilesh B
NGNLab, Department of Computer Technology, Anna University, MIT Campus, Chennai 600044, India

***Abstract*—Network Digital Twins (NDTs) enable operators to test modifications to an Open RAN (O-RAN) without impacting live traffic. A twin is only of use whilst its predictions remain in line with the actual network. As traffic and configuration change, the models within the twin will drift, and current standards provide no definition as to how to measure the twin's accuracy or what to do when it is no longer accurate. This paper presents a self-adaptive NDT for O-RAN which continuously measures its own fidelity and can repair itself. A single fidelity score is calculated for each twin component, combining four complementary measures of error and distribution. If a component's score remains low, then a multi-agent layer, led by a large language model (LLM) and grounded in a retrieval-augmented generation (RAG) knowledge base, will diagnose the root cause and select one of three possible repairs: recalibration, regeneration or scope expansion. Each candidate repair is tested in an isolated sandbox and only deployed if a deterministic acceptance gate confirms that it has improved fidelity. The LLM is thus never in a position to force a change into production. The framework is evaluated on OpenAirInterface 5G testbed data and is shown to achieve higher fidelity than that of a scheduled retrain, using around 3% of its compute. When compared to a rule-based repair policy, it requires 4 rather than 7 repair attempts, increases the acceptance rate from 29% to 75%, and halves both LLM usage and repair time.**

***Index Terms*—Network digital twin, O-RAN, fidelity, self-adaptive systems, large language models, retrieval-augmented generation, agentic AI.**

I. Introduction

An NDT is a continuously updated virtual representation of a live network (or indeed of a portion of the network such as the RAN). As it is constantly being updated with live data, it allows operators and automated controllers to assess the impact of configuration changes, upgrades and failure scenarios without any risk to production traffic [1]. For O-RAN, with its highly disaggregated, multi-vendor architecture and the constant stream of software updates that brings, offline testing followed by periodic updates is no longer viable and the twin has to be validated continuously [2].

The value of a twin is in its *fidelity*, i.e., how closely its behaviour mirrors that of the actual network. Fidelity is not a fixed state. Traffic mixes change, slices get reconfigured, controllers change their scheduling shares, and the models that have been learned by the twin will gradually drift from the data that they were trained on [3]. A low-fidelity twin will still give an answer, but it will be an answer that is confidently wrong.

Standardisation has moved on in terms of the "plumbing" of the NDT. 3GPP has defined the management objects and lifecycle APIs [4], ETSI has incorporated NDTs into its zero-touch management [5], and ITU-T specifies reference architectures and capability levels [1], [6]. None of these documents define a quantitative fidelity metric for a twin, nor the decision logic as to when a degraded twin should be recalibrated, regenerated or extended. In practice, twin maintenance is still very much dominated by periodic retraining, fixed thresholds and manual intervention.

This paper closes that gap with an autonomous, fidelity-driven maintenance framework for an O-RAN NDT. The main contributions are:

- A deterministic composite fidelity score per twin component that fuses four complementary point-error and distribution metrics, with a debounced per-component trigger.
- A knowledge-grounded decision and root cause analysis (RCA) agent which selects from amongst three graduated repair strategies and learns from failed attempts.
- A governed adaptation path in which every candidate (including LLM-generated code) is sandboxed and accepted only by a deterministic gate, with versioned promotion and rollback.
- An evaluation on OpenAirInterface (OAI) 5G testbed data against static, periodic and rule-based twins.

The rest of the paper is organised as follows: Section II reviews the related work, Section III presents the system model, Section IV contains the results, and Section V concludes the paper.

II. Related Work

Network digital twins and fidelity. Mirzaei *et al.* look at the enablers and use cases of NDTs for O-RAN [2]. Colosseum has built an O-RAN twin on large-scale RF channel emulation [7], a physically accurate ray-traced twin of an indoor 5G/6G testbed is validated against over-the-air measurements [8], and a 5G core twin is able to reproduce control-plane signalling [9]. Fidelity is reported in all of these cases in a very inconsistent manner. In the service-oriented attention HAPPO framework for xApp coordination, He *et al.* [10] train the xApps against a learned AI-RAN twin, measure RMSE, MAE, Wasserstein distance and MMD individually, and then fine-tune the twin when the error becomes too large. A survey

TABLE I
FIDELITY MEASUREMENT IN REPRESENTATIVE DIGITAL TWINS

System	How fidelity is measured	Single score
HAPPO AI-RAN twin [10]	RMSE, MAE, W_1, MMD (separate)	No
O-RAN twin toward 6G [11]	Coarse/fine tiers, sync latency	No
Resilient 6G RAN twin [12]	Sync speed, simulation detail	No
Ray-traced 5G/6G twin [8]	OTA RSRP, coverage, throughput	No
Agentic DTN [15]	KPI targets met (empirical)	No
6G NDT architecture [13]	RMSE, MSE, R^2 (separate)	No
ITU-T Y.3091 [6]	Qualitative capability levels	No
Traffic-monitoring NDT [14]	RMSE, MAE	No
5GC control-plane twin [9]	F1-score	No
Colosseum [7]	Emulation realism	No
This work	RMSE, MAE, W_1, MK-MMD, normalised and fused	**Yes**

of O-RAN twins classifies fidelity into coarse and fine-grained levels [11], [12], a 6G NDT reference architecture makes use of several separate KPIs (RMSE, MSE and R^2) [13], a traffic-monitoring twin equates fidelity with RMSE and MAE [14], and ITU-T treats fidelity purely qualitatively [6]. Table I summarises this landscape. To our knowledge, no existing twin produces a comparable normalised fidelity score per component in every evaluation cycle, and none of them tie such a score to a controlled selection of repair strategies.

Self-adaptive systems. Early work on adaptive software made the distinction between parameter adaptation and structural adaptation [16], and the self-adaptive systems roadmap went on to state that structural adaptation is compositional [17] (the replacement of a component being distinct from the addition of a new component). The concept drift literature makes a distinction between blind (scheduled) and informed (trigger-based) adaptation [3]. A self-adaptive NDT for transport networks detects drift and then retrains its twin [18].

LLM agents for the RAN. GENESIS employs AI agents that write, test and repair RAN code, with every answer grounded in a knowledge base populated with the results of previous runs [19]. ALLSTaR generates scheduler code, tests it and stores the tested solutions for re-use [20]. An agentic framework for digital twin networks enables LLM agents to design and deploy changes, albeit without an independent non-LLM veto [15]. Our framework combines such a knowledge-grounded agent with a deterministic acceptance gate which it cannot override.

III. SYSTEM MODEL

The proposed framework maintains an O-RAN NDT via a closed loop of monitoring, decision, repair, verification and recording. This section details the architecture, the fidelity model, and the rule by which a repaired model is accepted or rejected.

A. Architecture Overview

Fig. 1 shows the architecture. Telemetry from the OAI 5G testbed is validated and continuously synchronised into the twin state. A set of dependency-ordered prediction components predict seven RAN KPIs: downlink and uplink block error rate, PRB utilisation and throughput, and downlink RLC delay. A fidelity engine scores each component, and a per-component trigger passes any degraded components to the adaptation layer.

The adaptation layer consists of six agents: (i) a decision and RCA agent; (ii)–(iv) recalibration, regeneration and scope-expansion agents; (v) a verification agent; and (vi) a lifecycle agent which logs all adaptations. *Recalibration* re-trains an existing component on the most recent data (parameter adaptation). *Regeneration* has the LLM rewrite the component (structural replacement). *Scope expansion* has the LLM add a component to model behaviour that the twin currently does not model (structural extension) [16], [17].

The decision agent is backed by a read-only RAG knowledge base containing O-RAN documentation, twin documentation, adaptation policies and historical adaptation records. Past outcomes thus inform future decisions without any need for model retraining, in the spirit of GENESIS and ALLSTaR [19], [20]. The agent returns a strategy from a closed set along with a root cause explanation, a confidence value and the knowledge it used.

B. Fidelity Model

Fidelity is computed deterministically (no LLM can define or change it). For a component c with predictions $\hat{\mathbf{y}} = (\hat{y}_1, \ldots, \hat{y}_n)$ and ground truth $\mathbf{y} = (y_1, \ldots, y_n)$ over an evaluation window, four complementary metrics are computed. The root mean square error and the mean absolute error measure point accuracy:

$$\mathrm{RMSE} = \Big(\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2\Big)^{1/2}, \quad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|. \tag{1}$$

RMSE punishes occasional large errors, whereas MAE is a measure of the average error. Point errors by themselves, though, do not reveal whether the twin is actually reproducing the realistic patterns of network behaviour, which is what a what-if analysis is all about. The engine therefore also compares the distributions using the first Wasserstein distance:

$$W_1 = \int_0^1 \big|F_{\hat{y}}^{-1}(q) - F_{y}^{-1}(q)\big|\, dq, \tag{2}$$

where $F_{\hat{y}}^{-1}$ and F_{y}^{-1} are the empirical quantile functions of the predictions and the ground truth, respectively. W_1 captures shifts in location and scale. To capture differences in distribution shape (e.g., multi-modality), the multi-kernel maximum mean discrepancy (MK-MMD) is employed:

$$\mathrm{MMD}^2 = \mathbb{E}[k(y, y')] + \mathbb{E}[k(\hat{y}, \hat{y}')] - 2\,\mathbb{E}[k(y, \hat{y})], \tag{3}$$

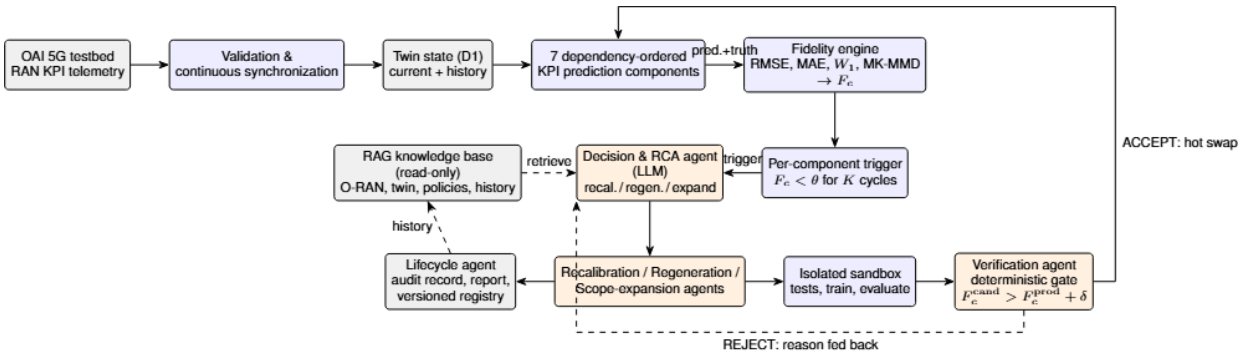


Fig. 1. Architecture of the self-adaptive O-RAN digital twin. Orange boxes are agents, blue boxes are deterministic components and grey boxes are data stores. Dashed arrows indicate knowledge retrieval and feedback.

with the kernel

$$k(x,x') = \sum_{j=1}^{m} \beta_j \exp\!\big(-\gamma_j (x-x')^2\big), \quad \beta_j \geq 0, \;\; \sum_{j=1}^{m} \beta_j = 1, \tag{4}$$

where the γ_j are Gaussian bandwidths centred on a median-heuristic base value so that both coarse and fine differences are detected. MMD^2 is estimated using the unbiased U-statistic and clipped at zero. The same four metrics are employed by recent O-RAN twin work [10] so as to keep our results comparable.

As the four metrics have different units and scales, we square each one ($D_m = m^2$ for $m \in \mathcal{M} = \{\mathrm{RMSE}, \mathrm{MAE}, W_1, \mathrm{MMD}\}$) and then min–max normalise it against a rolling reference window H_m of its L most recent values:

$$\tilde{D}_m = \frac{D_m - \min H_m}{\max H_m - \min H_m + \epsilon}, \tag{5}$$

where ϵ is a small stabilising constant. Squaring highlights the poor windows, and the reference window contains only past values, so a score never grades itself. The composite fidelity score for component c is

$$F_c = 1 - \frac{1}{4} \sum_{m \in \mathcal{M}} \tilde{D}_m, \tag{6}$$

which approaches 1 when the current error is at the low end of its recent history and approaches 0 when it is at the high end. If the reference window is too short, or the evaluation window is empty or contains non-finite values, then F_c is reported as *insufficient history* rather than being imputed. For system-level reporting, the unified score is the aggregate over the set $\mathcal{C}^*$ of components with a defined score:

$$F_U = \frac{\sum_{c \in \mathcal{C}^*} w_c F_c}{\sum_{c \in \mathcal{C}^*} w_c}, \tag{7}$$

where w_c are the configured weights (equal by default). F_U is a reporting signal only. Adaptation is driven by each component's own F_c, so a single failing component cannot be masked by healthy ones, and a healthy component will never be modified as a result of another component degrading.

C. Fidelity-Driven Trigger

With threshold θ and debounce length K, component c triggers an adaptation at evaluation cycle t if

$$F_c(t-k) < \theta \quad \forall\, k = 0, \ldots, K-1, \tag{8}$$

so that a single noisy cycle cannot start a repair. The severity of the degradation is

$$\sigma_c = \mathrm{clip}\Big(\frac{\theta - F_c}{\theta},\, 0,\, 1\Big), \tag{9}$$

which is 0 at the threshold and 1 at complete loss of fidelity. The trigger $\langle c, \sigma_c, t \rangle$ and the scores of all components are passed, along with the retrieved knowledge, to the decision agent. We use $\theta = 0.5$ and $K = 3$.

D. Knowledge-Grounded Decision Agent

For each trigger, a context builder constructs a bounded decision context: the scores of all the components, the triggered component and its severity, the previous action taken on that component and its outcome (including any rejection reason from the gate), and the most relevant passages retrieved from the knowledge base. For each trigger, one LLM call is made with a structured output schema. The LLM returns the selected strategy, a root cause analysis, a confidence, a rationale and references to the knowledge base passages it used. The output is validated against the schema, and a strategy outside the closed set of three is treated as a validation failure. The agent only falls back to recalibration if the LLM is unreachable or if validation fails repeatedly, and every such fallback is logged. The retrieved text is treated as untrusted data and is kept separate from the system instructions.

Earlier designs which learned the strategy via reinforcement learning were abandoned, as adaptation events are so rare that a learned policy receives too few reward samples and its decisions are very hard to explain to operators. The knowledge-grounded agent instead produces an explicit, auditable root cause analysis, and all of the numerical decisions are deterministic. At most one adaptation is applied to a component at any one time; triggers which arrive for a component already being adapted are merged, and telemetry ingestion and the production models continue to run.

E. Adaptation Process

The three strategies differ in how much of the component they actually alter. All of them construct the candidate from a recent window W of twin history, split in time into a training portion and a held-out evaluation portion:

$$W_{\mathrm{tr}} = \{x_i \in W : i \le \lfloor \rho |W| \rfloor\}, \qquad W_{\mathrm{ev}} = W \setminus W_{\mathrm{tr}}, \quad (10)$$

where $\rho = 0.8$. As the split is by time, no later sample is used to train the candidate that is subsequently evaluated on it.

Recalibration keeps the component's model class and retrains its parameters Θ_c. The recent samples $\mathcal{D}_{\mathrm{rec}} \subseteq W_{\mathrm{tr}}$ are combined with the original training data $\mathcal{D}_{\mathrm{ref}}$ and are given greater weight:

$$\Theta_c^{\mathrm{new}} = \arg\min_{\Theta} \sum_{i \in \mathcal{D}_{\mathrm{ref}} \cup \mathcal{D}_{\mathrm{rec}}} \omega_i \, \ell\big(f_c(x_i;\Theta), y_i\big), \quad (11)$$

with weights $\omega_i = \lambda$ for $i \in \mathcal{D}_{\mathrm{rec}}$ and $\omega_i = 1$ for $i \in \mathcal{D}_{\mathrm{ref}}$, where f_c is the component model, ℓ is the squared-error loss, and $\lambda > 1$ is the recency weight ($\lambda = 5$). The model thus adopts the new behaviour without discarding what it had previously learned, which is important when only a few hundred recent samples are available.

Regeneration replaces the implementation of the component whilst preserving its interface. The LLM is given the current source code, the error patterns, the feature statistics, the root cause analysis and the retrieved knowledge, and it writes a new model which is then trained on W_{tr}. If the sandbox rejects the code, the reason for rejection is fed back to the LLM for another iteration.

Scope expansion adds a new component c' for a KPI that the twin does not currently predict. The LLM proposes the inputs, output and dependencies $P(c')$, and the extended dependency graph $G' = (V \cup \{c'\},\ E \cup \{(a, c') : a \in P(c')\})$ must remain acyclic before the component is implemented and trained. This differs from retraining in that it adds a capability that the twin did not previously have [17].

F. Verification and Acceptance Gate

Every candidate produced by an adaptation agent (including LLM-generated code) is run only in an isolated sandbox with no write access to production code, models, registry or configuration. It is run through syntax and import checks, unit tests, training and evaluation. The verification agent then recomputes the fidelity score of the current production model F_c^{prod} and of the candidate F_c^{cand} using (6) on the same held-out data and the same normalisation reference H_m, so that the two scores are directly comparable. The candidate is accepted only if

$$\mathrm{ACCEPT} \iff F_c^{\mathrm{cand}} > F_c^{\mathrm{prod}} + \delta \ \wedge \ \bigwedge_{j=1}^{J} \mathrm{check}_j = \mathrm{pass}, \quad (12)$$

where $\delta > 0$ is a margin which rejects any improvement within the realm of measurement noise, and the J mandatory checks ensure that the candidate loads, conforms to the component interface, passes its tests and produces finite outputs. Failure of any one of these conditions results in rejection. For a new component created via scope expansion (i.e., one with no production model), F_c^{prod} is replaced by the score of a persistence predictor. We use $\delta = 0.01$.

Algorithm 1 Fidelity-Driven Adaptation and Acceptance

Require: component c with trigger $\langle c, \sigma_c, t\rangle$; margin δ; attempts A

lock c; $h_c \leftarrow \emptyset$
for $n = 1$ **to** A **do**
  $(a, \mathrm{RCA}) \leftarrow \textsc{DecisionAgent}(\{F_c\}, \sigma_c, h_c, \mathrm{RAG})$
  $\hat{m} \leftarrow \textsc{AdaptationAgent}_a(c)$; run $\hat{m}$ in sandbox
  compute F_c^{cand} and F_c^{prod} on the same data and reference
  if $F_c^{\mathrm{cand}} > F_c^{\mathrm{prod}} + \delta$ **and** all checks pass **then**
    promote $\hat{m}$; keep previous version for rollback; **break**
  else
    reject $\hat{m}$; $h_c \leftarrow h_c \cup \{(a, \mathrm{reason})\}$ ▷ fed back to agent
  end if
end for
write lifecycle record; unlock c

The gate is the only route to production. The verification agent uses the LLM to explain why a candidate has passed or failed, but this explanation in no way affects the outcome of (12). An accepted candidate is promoted via an atomic pointer switch in a versioned registry, so that the prior version remains available for rollback. A rejected candidate has no effect on production at all; its rejection reason is fed back to the decision agent, which may opt for a different strategy, up to A attempts (Algorithm 1).

IV. Results and Analysis

The proposed framework was benchmarked against three baselines on held-out OAI testbed data, each of which is a version of the full system missing one of its abilities: a *static* twin, which never adapts; *periodic* retraining of all components on a fixed schedule without any verification (blind adaptation [3]); and a *rule-based* twin, which is otherwise identical to the proposed system except that the decision agent is replaced with a fixed severity rule (low $\rightarrow$ recalibrate, medium $\rightarrow$ regenerate, high $\rightarrow$ expand):

$$a_{\mathrm{rule}}(\sigma_c) = \begin{cases} \text{recalibrate}, & \sigma_c \le 0.3, \\ \text{regenerate}, & 0.3 < \sigma_c \le 0.7, \\ \text{expand scope}, & \sigma_c > 0.7. \end{cases} \quad (13)$$

A. Comparison Metrics

Each twin's runtime score (6) is normalised against its own history, so scores from different twins are not directly comparable. For the purpose of comparison, each evaluation window w of each twin is re-scored using the same formula against

TABLE II
COMPARISON WITH BASELINES (MEAN FIDELITY ON A COMMON SCALE; COSTS PER RUN)

Mode	Mean fid.	95% CI	Attempts	Acc.	η
Static	0.8196	[0.7730, 0.8584]	0	–	–
Periodic	0.8122	[0.7676, 0.8490]	42*	–	–
Rule-based	0.8174	[0.7722, 0.8558]	7	2	29%
Proposed	0.8155	[0.7708, 0.8525]	4	3	75%

*Deployed without verification. Proposed values are averaged over three runs.

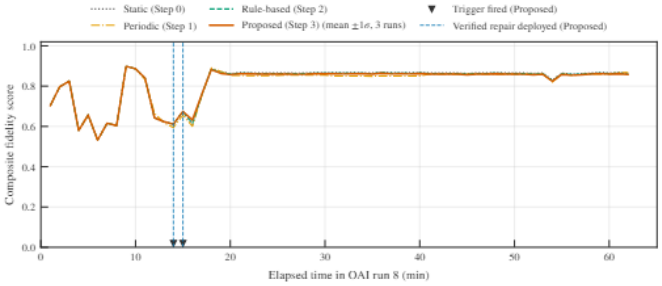


Fig. 2. Composite fidelity over time for the four twins. Triangles indicate triggers and dashed lines indicate verified repairs deployed by the proposed framework.

one frozen reference (the minimum $\underline{D}_{m,c}$ and maximum $\overline{D}_{m,c}$ of each squared metric, pooled over all twins):

$$\bar{F}(w) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \left(1 - \frac{1}{4} \sum_{m \in \mathcal{M}} \frac{D_{m,c}(w) - \underline{D}_{m,c}}{\overline{D}_{m,c} - \underline{D}_{m,c} + \epsilon} \right). \quad (14)$$

Two twins A and B are compared on the same windows through the mean paired difference

$$\bar{d} = \frac{1}{N} \sum_{w=1}^{N} \big(\bar{F}_A(w) - \bar{F}_B(w) \big), \quad (15)$$

over N windows. As neighbouring windows are correlated, the 95% CI is obtained via a moving block bootstrap (resampling blocks of consecutive windows), and the p-values from the M pairwise tests are Holm-adjusted as in [10]:

$$p_H(k) = \max_{j \le k} \min \big(1,\ (M - j + 1)\, p_{(j)} \big), \quad (16)$$

where $p_{(1)} \le \cdots \le p_{(M)}$ are the sorted raw p-values. The quality of repair decisions is measured by the acceptance rate and the recovery time:

$$\eta = \frac{N_{\text{acc}}}{N_{\text{att}}}, \qquad T_{\text{rec}} = \min\{t > t_0 : F_c(t) \ge \theta\} - t_0, \quad (17)$$

where N_{acc} and N_{att} are the numbers of accepted and attempted repairs and t_0 is the trigger time. For each repair, the compute time, adaptation time and LLM token usage are recorded. Table II shows the results.

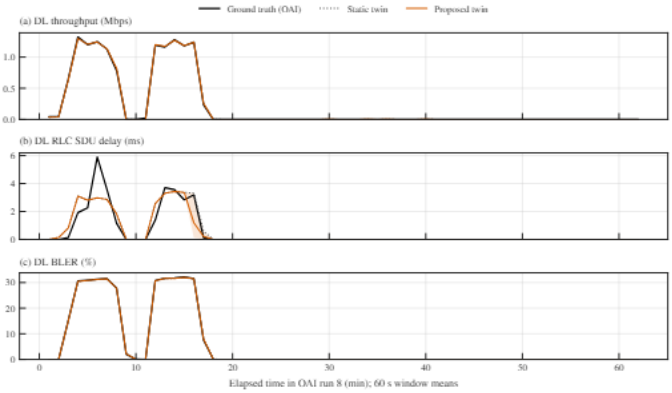


Fig. 3. Measured and predicted KPIs (60-s means): (a) DL throughput, (b) DL RLC delay and (c) DL BLER, for the static and proposed twins.

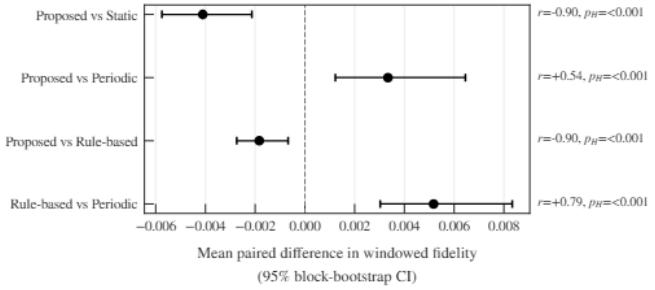


Fig. 4. Mean paired difference in windowed fidelity with 95% block-bootstrap CIs, effect sizes r and Holm-adjusted p-values.

B. Fidelity Tracking

Fig. 2 shows the composite fidelity of all four twins over time. All twins track the testbed closely, with mean fidelity between 0.812 and 0.820, and the three runs of the proposed system are nearly identical (standard deviation 1.8×10^{-4}). The fidelity-driven trigger fired only when components degraded, and in every mode the degraded components were back above the threshold within one to three minutes. Fig. 3 compares measured and predicted KPIs. Both twins follow the measured throughput and BLER closely. The shaded band in Fig. 3(b) marks the minute after the RLC delay repair, where the repaired model predicted the end of the traffic burst one minute early.

C. Comparison with Periodic Retraining

The proposed framework achieved higher fidelity than periodic retraining, with a mean paired difference of $+0.0033$ (95% block-bootstrap CI $[0.0012, 0.0065]$, Holm-adjusted $p < 0.001$). It did so with 137.5 CPU-s of adaptation compute versus 4779.3 CPU-s (around 3%), as shown in Fig. 5. Periodic retraining resulted in 42 unverified models being deployed and produced the largest distribution errors (normalised Wasserstein 0.090, MK-MMD 0.467): the retraining schedule shifted the predicted distribution away from the actual measured one. The acceptance gate stops such updates from reaching production. Fig. 4 shows all the paired comparisons with their confidence intervals.

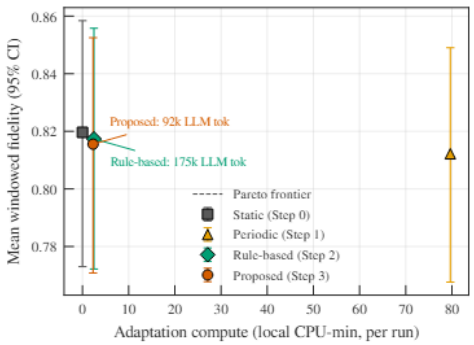


Fig. 5. Mean fidelity (95% CI) against adaptation compute per run.

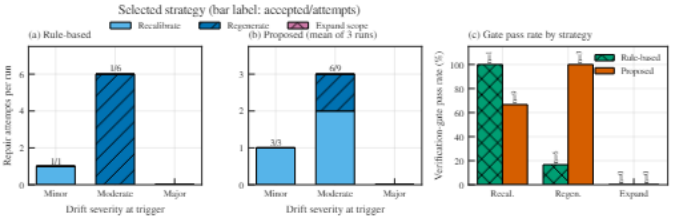


Fig. 6. Strategies selected by (a) the rule-based policy and (b) the proposed agent (bar labels: accepted/attempts), and (c) acceptance rate per strategy.

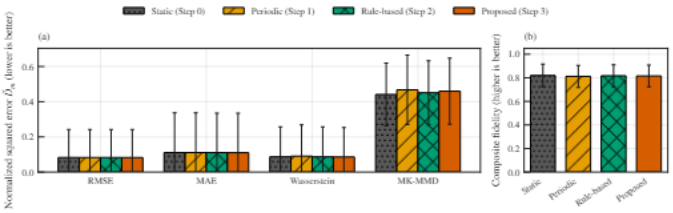


Fig. 7. (a) Mean normalised error per fidelity metric (lower is better) and (b) composite fidelity for each twin.

D. Comparison with Rule-Based Repair Selection

The rule-based and proposed twins share all components save for the strategy decision, so the difference between them isolates the value of the knowledge-grounded agent (Fig. 6). For the uplink BLER component, the severity rule dictated regeneration, and all 5 of the regenerated candidates were rejected by the gate, leaving the component unrepaired. The decision agent instead chose recalibration, which was accepted in all runs. For the RLC delay component, the agent initially selected recalibration, which was rejected; the feedback reason then led it to regeneration, which was accepted in all runs.

Overall, the proposed framework reduced the number of repair attempts from 7 to 4 per run, increased the acceptance rate from 29% to 75%, halved the LLM tokens and cost, and reduced adaptation time from 255.3 s to 116.4 s. It also restored the RLC delay component in under one minute in two of the three runs (vs. two minutes for each of the baselines). Periodic retraining deployed all 42 of its models unverified, whereas 9 of the 12 candidates generated in three runs of the proposed framework passed the gate.

E. Comparison with the Static Twin

On mean fidelity, the static twin was 0.0041 better than the proposed framework, and the rule-based twin 0.0018 better. All the modes are within 0.006 of each other. The proposed framework had lower normalised RMSE (0.0826 vs. 0.0828), lower MAE (0.1101 vs. 0.1108) and lower Wasserstein error (0.0858 vs. 0.0868) than the static twin. The difference is entirely due to MK-MMD (0.459 vs. 0.441), as can be seen in Fig. 7. Most of the evaluation period had no traffic, so there were very few degradation events for adaptation to correct. Evaluating on longer, more dynamic traffic is the main area for future work.

V. Conclusion

This paper presented a self-adaptive digital twin for O-RAN which measures its own fidelity with one deterministic score per component, only triggers repair when a component is in a degraded state, and uses a knowledge-grounded LLM agent to select between recalibration, regeneration and scope expansion. All candidates are sandboxed and only admitted through a deterministic acceptance gate. All changes are versioned, reversible and auditable. On the OAI testbed data, the framework achieved a 3% reduction in compute over scheduled retraining and reached accepted repairs with fewer attempts, a higher acceptance rate, and at half the LLM cost of a rule-based policy.

Future work will evaluate the framework on longer and more varied traffic, add forward validation to the acceptance gate, and extend the twin to end-to-end latency and multi-cell deployments.

References

[1] ITU-T, "Digital twin network – Requirements and architecture," Recommendation Y.3090, International Telecommunication Union, 2022.

[2] J. Mirzaei, I. Abualhaol, and G. Poitau, "Network digital twin for Open RAN: The key enablers, standardization, and use cases," *arXiv preprint arXiv:2308.02644*, 2023.

[3] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[4] 3GPP, "Management and orchestration; Management aspects of network digital twin," Technical Specification TS 28.561, 3rd Generation Partnership Project.

[5] ETSI, "Zero-touch network and service management (ZSM); Network digital twin for enhanced zero-touch network and service management," Group Specification GS ZSM 018, European Telecommunications Standards Institute.

[6] ITU-T, "Digital twin network – Capability levels and evaluation methods," Recommendation Y.3091, International Telecommunication Union.

[7] M. Polese *et al.*, "Colosseum: The Open RAN digital twin," *IEEE Open Journal of the Communications Society*, 2024.

[8] "Digital twin powered next generation wireless networks: Construction, validation, and applications."

[9] "Deep learning-based modeling of 5G core control plane for 5G network digital twin," 2023.

[10] Z. He, M. Shojafar, and R. Tafazolli, "Service-oriented attention HAPPO for xApps coordination in digital twin enabled AI-RAN," *IEEE Transactions on Network and Service Management*, 2026.

[11] "Digital twin for O-RAN toward 6G."

[12] "Digital twin for Open RAN: Toward intelligent and resilient 6G RAN."

[13] "A reference functional architecture for network digital twins in 6G systems."

[14] "From prediction to protection: Network digital twin for traffic monitoring," 2026.

[15] “Agentic-AI framework for integrated design, implementation, testing, and operation of digital twin networks.”
[16] P. K. McKinley, S. M. Sadjadi, E. P. Kasten, and B. H. C. Cheng, “Composing adaptive software,” *IEEE Computer*, vol. 37, no. 7, pp. 56–64, 2004.
[17] B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee, *et al.*, “Software engineering for self-adaptive systems: A research roadmap,” in *Software Engineering for Self-Adaptive Systems*, ser. Lecture Notes in Computer Science, vol. 5525. Springer, 2009.
[18] C. Modesto, J. Borges, C. Nahum, L. Matni, C. B. Both, K. Cardoso, G. Gonçalves, I. Correa, S. Lins, A. Silva, and A. Klautau, “Towards a robust transport network with self-adaptive network digital twin,” *Computer Networks*, 2025, arXiv:2507.20971.
[19] “GENESIS: Harnessing AI agents for autonomous 6G RAN synthesis, research, and testing.”
[20] “ALLSTaR: Automated LLM-driven scheduler generation and testing for intent-based RAN.”